

Data Sparsity Problem in N-gram LMs:

- Human language has a massive, combinatorial vocabulary, meaning that even huge text corpora contain only a tiny fraction of all valid word combinations.
- As a result, a large majority of perfectly natural, grammatical N -grams never appear in the training corpus, receiving an empirical count of zero ($C(w_{k-N+1:k-1}, w_k) = 0$).

- **Sequence Probability Collapse:** Because the joint probability of a sentence is computed as a product of conditional probabilities, a single unseen N -gram causes the probability of the entire sequence to drop to zero:

$$P(W) = \prod_{k=1}^n P(w_k \mid w_{k-N+1:k-1}) = 0$$

- **Perplexity Explosion:** Evaluating test data containing an unseen N -gram makes cross-entropy undefined ($\log_2(0) \rightarrow -\infty$) and drives perplexity to infinity ($PP(W) \rightarrow \infty$).
- A count of zero does not mean a phrase is impossible—it simply means the training sample was not large enough to observe it.

Solutions:

- **Smoothing (Discounting):** Reallocating probability mass from high-frequency N -grams to reserve a non-zero probability budget for unseen events (e.g., Laplace, Add- m , Good-Turing).
- **Back-off & Interpolation:** Falling back to or blending with lower-order models (such as backing off from trigrams to bigrams and unigrams), where counts are much denser and more reliable.
- **Continuous / Neural Representations:** Modern neural language models (RNNs, Transformers) mitigate this fundamentally by representing words in dense, continuous vector spaces (embeddings), where semantically similar contexts share statistical strength instead of relying on exact string matches.

Intuition of Smoothing:

- When we have sparse statistics:

Count(w | denied the)

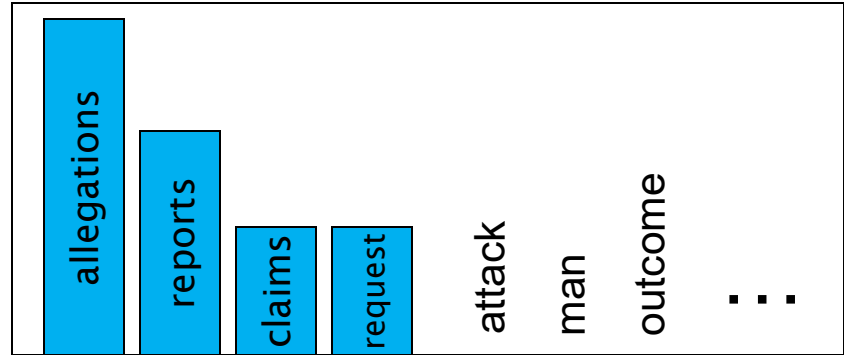
3 allegations

2 reports

1 claims

1 request

7 total



Steal probability mass to generalize better

Count(w | denied the)

2.5 allegations

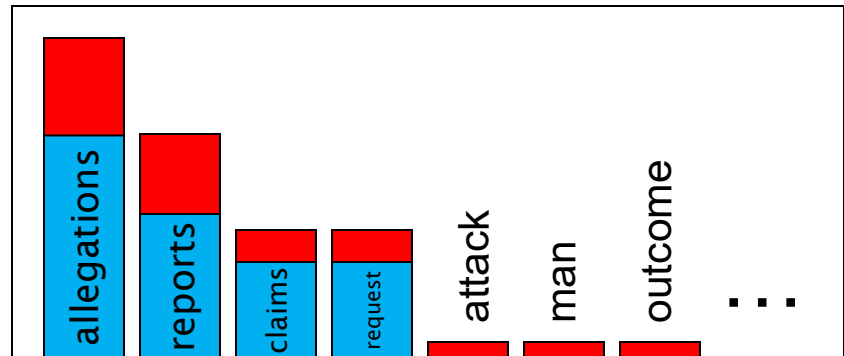
1.5 reports

0.5 claims

0.5 request

2 others

7 total



The Zero-Probability Problem: Solution -> Smoothing

- Smoothing solves this by shaving off a small amount of probability mass from frequent events and reallocating it to unseen events (zero-count N -grams).
- **Laplace (Add-1) Smoothing**
- Laplace smoothing pretends that every possible N -gram in the vocabulary has been observed **one extra time**.

- **Bigram Formula:**

- $$P_{\text{Laplace}}(w_k \mid w_{k-1}) = \frac{C(w_{k-1}, w_k) + 1}{C(w_{k-1}) + |V|}$$

- $C(w_{k-1}, w_k)$ is the bigram frequency.
 - $C(w_{k-1})$ is the history frequency.
 - $|V|$ is the total vocabulary size.

- **Why Add $|V|$ to the Denominator?**

- To maintain a valid probability distribution where $\sum_{w_k \in V} P(w_k \mid w_{k-1}) = 1$:

- $$\sum_{w_k \in V} \frac{C(w_{k-1}, w_k) + 1}{C(w_{k-1}) + |V|} = \frac{\sum_{w_i} C(w_{k-1}, w_k) + \sum_{w_i} 1}{C(w_{k-1}) + |V|} = \frac{C(w_{k-1}) + |V|}{C(w_{k-1}) + |V|} = 1$$

The Zero-Probability Problem: Solution -> Smoothing

- **Add- m (Lidstone) Smoothing**

- While Laplace smoothing prevents zero probabilities, adding an entire virtual count of 1 is far too aggressive when $|V|$ is large (e.g., $|V| = 50,000$). It shifts an overwhelming amount of probability mass from observed data to unseen pairs, severely degrading model accuracy.
- **Add- m smoothing** relaxes this by replacing the integer 1 with a fractional pseudo-count m (typically $0 < m < 1$, such as $m = 0.05$ or $m = 0.1$):
- $$P_{\text{Add-}m}(w_k \mid w_{k-1}) = \frac{C(w_{k-1}, w_k) + m}{C(w_{k-1}) + m|V|}$$
- **Tuning m :** The parameter m is treated as a hyperparameter and tuned on a **validation (held-out) set** to minimize perplexity before evaluating on the test set.
- **Behavior:** As $m \rightarrow 0$, the estimate approaches pure MLE. As $m \rightarrow 1$, it becomes standard Laplace smoothing.

Branching factor in N-gram LMs:

- In an N -gram language model, the **branching factor** represents the number of plausible next words the model is trying to choose among given a preceding history $(w_{k-N+1}, \dots, w_{k-1})$.
- Because human language distributions are non-uniform—some words are highly probable while others are nearly impossible—the branching factor is measured as an **effective branching factor**, which is mathematically identical to **perplexity** (PP):
- **Why perplexity equals the branching factor?**

Branching factor in N-gram LMs:

- **Uniform Uncertainty (Worst Case):**

- If a model has a vocabulary of size $|V| = 10,000$ and predicts that every word in V is equally likely ($P(w) = \frac{1}{10,000}$):
- $H = -\log_2 \left(\frac{1}{10,000} \right) = \log_2(10,000) \approx 13.29$ bits
- $PP = 2^H = 10,000$
- The effective branching factor is exactly 10,000—the model has zero context and makes a blind choice across the entire vocabulary.

- **Skewed / Informed Distribution (Realistic Case):**

- The model does not consider all 10,000 words equally; it concentrates high probability mass on some words with residual probability scattered among a few others.
- The resulting cross-entropy is $H < 13.29$ bits, the perplexity is: $PP < 10,000$
- **Interpretation:** Even though the vocabulary has 10,000 words, the model's uncertainty is equivalent to choosing uniformly among less equally likely words compared to uniform uncertainty case.

Example:

- Using the earlier corpus, here is the complete step-by-step numerical calculation of perplexity for a **Bigram ($n = 2$) model with Laplace (Add-1) smoothing**.
- **1. Training Corpus & Vocabulary Definition**
- **Corpus Sentences:**
 - $\langle s \rangle$ I am Sam $\langle /s \rangle$
 - $\langle s \rangle$ Sam I am $\langle /s \rangle$
 - $\langle s \rangle$ I do not like green eggs $\langle /s \rangle$
- **Vocabulary Extraction:**
 - Unique word types (V): $\{\langle s \rangle, \langle /s \rangle, \text{"I"}, \text{"am"}, \text{"Sam"}, \text{"do"}, \text{"not"}, \text{"like"}, \text{"green"}, \text{"eggs"}\}$
- **Vocabulary Size ($|V|$): 10**
- **2. Count Tables**
- **Unigram Counts $C(w)$:**
 - $C(\langle s \rangle) = 3$
 - $C(\langle /s \rangle) = 3$
 - $C(\text{I}) = 3$
 - $C(\text{am}) = 2$
 - $C(\text{Sam}) = 2$
 - $C(\text{do}) = 1$
 - $C(\text{not}) = 1$
 - $C(\text{like}) = 1$
 - $C(\text{green}) = 1$
 - $C(\text{eggs}) = 1$

Example:

- **Relevant Bigram Counts** $C(w_{k-1}, w_k)$:
 - $C(< s >, I) = 2$
 - $C(< s >, \text{Sam}) = 1$
 - $C(I, \text{am}) = 2$
 - $C(I, \text{do}) = 1$
 - $C(\text{am}, \text{Sam}) = 1$
 - $C(\text{am}, < / s >) = 1$
 - $C(\text{Sam}, < / s >) = 1$
 - $C(\text{Sam}, I) = 1$
 - All other bigrams = 0
- **3. Test Sequence to Evaluate**
 - Let the test sequence W be the sentence:
 - $< s > I \text{ am Sam } < / s >$
 - Tokens to predict: $w_1 = I$, $w_2 = \text{am}$, $w_3 = \text{Sam}$, $w_4 = < / s >$
 - **Total test tokens (n):** 4 (standard convention includes the end-of-sentence marker $< / s >$, while $< s >$ serves as initial conditioning context).

Example:

- **4. Laplace (Add-1) Smoothed Probability Estimation**

- The Add-1 smoothed probability formula is:

- $$P_{Laplace}(w_k | w_{k-1}) = \frac{C(w_{k-1}, w_k) + 1}{C(w_{k-1}) + |V|}$$

- Where $|V| = 10$.

- **Step 1:** $P(l | <s>)$

- $$P(l | <s>) = \frac{C(<s>, l) + 1}{C(<s>) + 10} = \frac{2 + 1}{3 + 10} = \frac{3}{13} \approx 0.23077$$

- **Step 2:** $P(am | l)$

- $$P(am | l) = \frac{C(l, am) + 1}{C(l) + 10} = \frac{2 + 1}{3 + 10} = \frac{3}{13} \approx 0.23077$$

- **Step 3:** $P(Sam | am)$

- $$P(Sam | am) = \frac{C(am, Sam) + 1}{C(am) + 10} = \frac{1 + 1}{2 + 10} = \frac{2}{12} = \frac{1}{6} \approx 0.16667$$

- **Step 4:** $P(</s> | Sam)$

- $$P(</s> | Sam) = \frac{C(Sam, </s>) + 1}{C(Sam) + 10} = \frac{1 + 1}{2 + 10} = \frac{2}{12} = \frac{1}{6} \approx 0.16667$$

- **5. Total Sequence Probability $P(W)$**

- $$P(W) = P(l | <s>) \times P(am | l) \times P(Sam | am) \times P(</s> | Sam)$$

- $$P(W) = \frac{3}{13} \times \frac{3}{13} \times \frac{1}{6} \times \frac{1}{6} = \frac{9}{169 \times 36} = \frac{9}{6084} = \frac{1}{676} \approx 0.00147929$$

Example:

- **6. Perplexity Calculation**

- Using the direct formula $PP(W) = P(W)^{-\frac{1}{n}}$ where $n = 4$:

- $PP(W) = \left(\frac{1}{676}\right)^{-\frac{1}{4}} = (676)^{\frac{1}{4}} = \sqrt{\sqrt{676}} = \sqrt{26} \approx 5.099$

- **7. Alternative Calculation via Log Cross-Entropy (Base 2)**

- $\log_2 P(l | \langle s \rangle) = \log_2(3/13) \approx -2.11547$

- $\log_2 P(am | l) = \log_2(3/13) \approx -2.11547$

- $\log_2 P(\text{Sam} | am) = \log_2(1/6) \approx -2.58496$

- $\log_2 P(\langle /s \rangle | \text{Sam}) = \log_2(1/6) \approx -2.58496$

- **Cross-Entropy $H(W)$:**

- $H(W) = -\frac{1}{4}((-2.11547) + (-2.11547) + (-2.58496) + (-2.58496)) =$
 $-\frac{1}{4}(-9.40086) \approx 2.35022 \text{ bits/word}$

- **Perplexity:**

- $PP(W) = 2^{H(W)} = 2^{2.35022} \approx 5.099$

- The model exhibits an effective branching factor of **~5.10** on this test sequence.

The Zero-Probability Problem: Solution -> Back-off & Interpolation

- In N -gram language modeling, both **back-off** and **interpolation** address data sparsity and the zero-probability problem by combining evidence from higher-order N -grams (which offer rich context but are sparse) with lower-order N -grams (which have weaker context but are frequently observed).
- **1. Interpolation**
- In **interpolation**, the model **always blends** probability estimates from all orders (e.g., trigram, bigram, and unigram), regardless of whether the higher-order N -gram was observed in the training corpus.
- **Linear Interpolation Formula (Trigram):**
- $\hat{P}(w_k \mid w_{k-2}, w_{k-1}) = \lambda_3 P_{MLE}(w_k \mid w_{k-2}, w_{k-1}) + \lambda_2 P_{MLE}(w_k \mid w_{k-1}) + \lambda_1 P_{MLE}(w_k); \quad \sum_{j=1}^3 \lambda_j = 1 \quad \text{where } \lambda_j \geq 0$
- **Setting the Weights:**
 - **Static Weights:** A fixed $\lambda_1, \lambda_2, \lambda_3$ tuned directly on a held-out validation set (often using the Expectation-Maximization algorithm) to minimize validation perplexity.
 - **Context-Dependent Weights:** Letting λ depend on context frequency, $\lambda(w_{k-2}, w_{k-1})$. If the history context (w_{k-2}, w_{k-1}) appears frequently in training, λ_3 is assigned a higher weight; if the context is rare, λ_3 is downscaled in favour of lower-order estimates.

The Zero-Probability Problem: Solution -> Back-off & Interpolation

- 2. Back-off

- In **back-off**, the model uses the highest-order N -gram if it has sufficient evidence. It falls back to lower-order models **only when the higher-order count is zero (or below a threshold)**.
- To make room for lower-order probabilities, the higher-order MLE counts must be **discounted** (using methods like Absolute Discounting or Good-Turing), freeing up a reserved probability mass that is redistributed to the fallback models.
- **Katz Back-off Formula (Trigram to Bigram):**

- $$P_{\text{BO}}(w_k \mid w_{k-2}, w_{k-1}) = \begin{cases} P^*(w_k \mid w_{k-2}, w_{k-1}) & \text{if } C(w_{k-2}, w_{k-1}, w_k) > 0 \\ \alpha(w_{k-2}, w_{k-1}) \cdot P_{\text{BO}}(w_k \mid w_{k-1}) & \text{if } C(w_{k-2}, w_{k-1}, w_k) = 0 \end{cases}$$

- **Where:**

- $P^*(w_k \mid w_{k-2}, w_{k-1}) = \frac{C(w_{k-2}, w_{k-1}, w_k) - d}{C(w_{k-2}, w_{k-1})}$ is the discounted probability estimate (d is the discount parameter, typically $0 < d < 1$).
- $\alpha(w_{k-2}, w_{k-1})$ is the **normalization / back-off weight**, ensuring that the remaining probability mass handed down to the lower order sums to exactly what was subtracted by discounting:
- $$\alpha(w_{k-2}, w_{k-1}) = \frac{1 - \sum_{w: C(w_{k-2}, w_{k-1}, w) > 0} P^*(w \mid w_{k-2}, w_{k-1})}{\sum_{w: C(w_{k-2}, w_{k-1}, w) = 0} P_{\text{BO}}(w \mid w_{k-1})}$$